Chapter 22 | Exercise solutions with Python

Object Detection and Segmentation

This notebook solves Exercises 5–8. It measures overlaps for boxes and masks, inspects a one-to-one assignment, and follows detections in confidence order at two overlap thresholds.

Each exercise uses the chapter's local NumPy implementations and checks the returned values against a small hand calculation. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch22_detection"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check touching edges

Place box A at $(0, 0, 2, 3)$ and box B at $(2, 0, 4, 3)$. They share the vertical edge at $x = 2$. Move B a quarter unit left to $(1.75, 0, 3.75, 3)$, and the shared region becomes a thin rectangle.

The coordinates (x_0, y_0, x_1, y_1) mark continuous edges. The width is $x_1 - x_0$, and the height is $y_1 - y_0$. Both boxes have area $2 \cdot 3 = 6$ before and after the shift. The intersection width comes from the smaller right edge minus the larger left edge, clamped at zero. The intersection height is 3 in both cases.

- When the edges touch, the overlap width is $2 - 2 = 0$. The common edge has zero area, so the union is $6 + 6 = 12$ and IoU is $0/12 = 0$.
- After the shift, the overlap width is $2 - 1.75 = 0.25$. The intersection area is $0.25 \cdot 3 = 0.75$, and the union area is $6 + 6 - 0.75 = 11.25$. The IoU is $0.75/11.25 = 1/15 \approx 0.066667$.

```
from examples.ch22_detection.model import box_iou

box_a = (0, 0, 2, 3)
box_b_touching = (2, 0, 4, 3)
box_b_shifted = (2 - 0.25, 0, 4 - 0.25, 3)

touching_iou = box_iou(box_a, box_b_touching)
shifted_iou = box_iou(box_a, box_b_shifted)

# Check the implementation against the areas from the hand calculation.
cases = [
    ("Touching", touching_iou, 0 * 3),
    ("Shifted left", shifted_iou, (2 - 1.75) * 3),
]
print(f"{'Case':<14} {'Intersection':>12} {'Union':>7} {'IoU':>10}")
for label, observed_iou, intersection in cases:
    union = 6 + 6 - intersection
    np.testing.assert_allclose(observed_iou, intersection / union)
    print(f"{label:<14} {intersection:>12.2f} {union:>7.2f} "
          f"{observed_iou:>10.6f}")
```

```
OUTPUT
Case           Intersection   Union        IoU
Touching               0.00   12.00   0.000000
Shifted left           0.75   11.25   0.066667
```

The shift leaves both widths at two units and creates a positive overlap of exactly a quarter unit in width. The continuous-edge convention treats each endpoint as a boundary, so the width formula has no extra unit for an endpoint.

6. Inspect an empty union (optional)

Start with two 3×3 Boolean masks on the same pixel grid. Every entry is `False`, so neither mask marks any foreground. Add a foreground pixel at row 1, column 1 of the second mask.

The intersection counts pixels marked by both masks. The union counts pixels marked by either mask, counting a shared pixel once.

- With both masks empty, the intersection and union both contain zero pixels. The ratio $0/0$ is undefined, and `mask_iou` returns `None`.
- With one marked pixel in the second mask, the intersection is still empty and the union contains that one pixel. The ratio $0/1 = 0$ is a defined score for zero shared foreground.

```
from examples.ch22_detection.model import mask_iou

empty_mask_a = np.zeros((3, 3), dtype=bool)
empty_mask_b = np.zeros((3, 3), dtype=bool)
both_empty_iou = mask_iou(empty_mask_a, empty_mask_b)

one_pixel_mask_b = empty_mask_b.copy()
one_pixel_mask_b[1, 1] = True
one_pixel_iou = mask_iou(empty_mask_a, one_pixel_mask_b)

# A is empty, so B supplies every pixel in the union.
empty_union_count = np.count_nonzero(empty_mask_b)
one_pixel_union_count = np.count_nonzero(one_pixel_mask_b)
assert empty_union_count == 0 and both_empty_iou is None
assert one_pixel_union_count == 1 and one_pixel_iou == 0 / 1

print(f"{'Case':<14} {'Intersection':>12} {'Union':>7} {'IoU':>8}")
print(f"{'Both empty':<14} {0:>12} {empty_union_count:>7} "
      f"{str(both_empty_iou):>8}")
print(f"{'One pixel':<14} {0:>12} {one_pixel_union_count:>7} "
      f"{one_pixel_iou:>8.1f}")
```

```
OUTPUT
Case           Intersection   Union      IoU
Both empty                0       0     None
One pixel                 0       1      0.0
```

The first pair has no foreground over which to measure overlap. The second pair does have foreground to compare, and the empty first mask shares none of it. The two return values preserve that difference.

7. Inspect matching indices

Use the three query rows from Exercise 3. The rows are Q1, Q2, and Q3, in that order. The columns are targets A and B. Both column minima pick Q1, for costs 2 and 1, so using those two minima would assign the same query twice.

Try each pair of distinct queries. The total for Q1 with A and Q3 with B, for example, is $2 + 2 = 4$. Increasing entry `[0, 1]` from 1 to 5 changes only the totals that pair Q1 with B.

Query for A	Query for B	Original total	Revised total
Q1	Q2	7	7
Q1	Q3	4	4
Q2	Q1	4	8
Q2	Q3	5	5
Q3	Q1	5	9
Q3	Q2	9	9

The original matrix has two equally good assignments. The helper tries query indices in ascending order for A, then for B. It keeps the first minimum, Q1 for A and Q3 for B, and reports zero-based `(query, target)` pairs, so this is `((0, 0), (2, 1))` and query index 1 (Q2) is unmatched. The other minimum, Q2 for A and Q1 for B, rises from 4 to 8. The returned assignment stays at cost 4 and becomes the unique minimum.

```
from examples.ch22_detection.model import match_queries

original_cost = np.array([[2, 1], [3, 5], [4, 2]])
revised_cost = original_cost.copy()
revised_cost[0, 1] += 4

original_match = match_queries(original_cost)
revised_match = match_queries(revised_cost)

for label, result in (
    ("Original", original_match), ("Revised", revised_match)
):
    pairs, unmatched, total = result
    print(f"{label} pairs {pairs}")
    print(f"  Unmatched {unmatched}    Total cost {total}")

expected_match = (((0, 0), (2, 1)), (1,), 4)
assert original_match == expected_match
assert revised_match == expected_match
assert original_cost[1, 0] + original_cost[0, 1] == 4
assert revised_cost[1, 0] + revised_cost[0, 1] == 8
```

OUTPUT

```
Original pairs ((0, 0), (2, 1))
  Unmatched (1,)    Total cost 4
Revised pairs ((0, 0), (2, 1))
  Unmatched (1,)    Total cost 4
```

8. Change the overlap threshold

Take the two annotated boxes $(0, 0, 3, 3)$ and $(5, 0, 8, 3)$ as targets A and B. All boxes belong to the same class in one image. The prediction with score 0.90 is $(1, 0, 4, 3)$, a shifted version of A. Its shared rectangle with A is two units wide and three high. Each box has area 9, so their intersection over union (IoU) is

$$\text{IoU} = \frac{2 \cdot 3}{9 + 9 - 2 \cdot 3} = \frac{6}{12} = 0.50.$$

The 0.80 and 0.70 predictions are exact copies of A and B. The evaluator sorts predictions by descending score. For each prediction it checks the unused annotation with the highest IoU and accepts a match when the IoU is **at least** the threshold. Each annotation can be used once.

- At 0.50 the first prediction matches A exactly at the threshold. The second prediction is a duplicate because A is already used. It has zero overlap with B. The third prediction matches B.
- At 0.75 the first prediction falls below the threshold, so A is still available. The second prediction matches A, and the third prediction matches B.

After each prediction, precision divides the number of true positives so far by the number of predictions seen. Recall divides the same count by the two annotations. For example, by rank two each threshold has one true positive, so both fractions are 1/2. The order in which the matches occur changes the curve.

```
from examples.ch22_detection.model import (
    all_points_ap,
    box_iou,
    precision_recall,
)

annotations = np.array([[0, 0, 3, 3], [5, 0, 8, 3]])
predictions = np.array([[1, 0, 4, 3], [0, 0, 3, 3], [5, 0, 8, 3]])
scores = np.array([0.90, 0.80, 0.70])

assert box_iou(predictions[0], annotations[0]) == 0.50

expected_by_threshold = {
    0.50: ([True, False, True], [1, 1/2, 2/3], [1/2, 1/2, 1]),
    0.75: ([False, True, True], [0, 1/2, 2/3], [0, 1/2, 1]),
}
```

```
curves = {}
for threshold, expected in expected_by_threshold.items():
    order, flags, precision, recall = precision_recall(
        annotations, predictions, scores, threshold=threshold
    )
    expected_flags, expected_precision, expected_recall = expected
    np.testing.assert_array_equal(order, [0, 1, 2])
    np.testing.assert_array_equal(flags, expected_flags)
    np.testing.assert_allclose(precision, expected_precision)
    np.testing.assert_allclose(recall, expected_recall)
    curves[threshold] = (precision, recall)

    print(f"IoU threshold {threshold:.2f}")
    print(f"  Ranked indices      {order.tolist()}")
    print(f"  True-positive flags {flags.tolist()}")
    print(f"  Precision           {precision.round(3).tolist()}")
    print(f"  Recall              {recall.tolist()}")
```

OUTPUT

```
IoU threshold 0.50
  Ranked indices      [0, 1, 2]
  True-positive flags [True, False, True]
  Precision           [1.0, 0.5, 0.667]
  Recall              [0.5, 0.5, 1.0]
IoU threshold 0.75
  Ranked indices      [0, 1, 2]
  True-positive flags [False, True, True]
  Precision           [0.0, 0.5, 0.667]
  Recall              [0.0, 0.5, 1.0]
```

To compute all-points average precision (AP), take the best precision achieved at each recall level or higher. This gives a step function called the *precision envelope*. Its area is the sum of each recall increase times the envelope height on that interval. A rank that leaves recall unchanged adds zero width.

At 0.50 the envelope is 1 over the first half of recall and 2/3 over the second half. At 0.75 the first prediction gives zero recall. At rank two the recall is 1/2 and precision is 1/2, but the later point at full recall improves precision to 2/3. That later value sets the envelope height over both halves. Thus

$$\begin{aligned} AP_{0.50} &= \tfrac{1}{2}(1) + \tfrac{1}{2}(\tfrac{2}{3}) = \tfrac{5}{6}, \\ AP_{0.75} &= \tfrac{1}{2}(\tfrac{2}{3}) + \tfrac{1}{2}(\tfrac{2}{3}) = \tfrac{2}{3}. \end{aligned}$$

Both thresholds reach full recall. AP is lower by 1/6 at 0.75 because the highest-scored prediction fails to match.

```
for threshold, expected_ap in ((0.50, 5/6), (0.75, 2/3)):
    precision, recall = curves[threshold]
    ap = all_points_ap(precision, recall)
    np.testing.assert_allclose(ap, expected_ap)
    print(f"All-points AP at IoU {threshold:.2f} = {ap:.6f}")
```

OUTPUT

```
All-points AP at IoU 0.50 = 0.833333
All-points AP at IoU 0.75 = 0.666667
```